ERP Frontend Redesign and Laravel Integration

Comprehensive architecture and implementation report

Prepared for the product, design, engineering, QA and operations teams | 1 November 2026

Executive recommendation

Undertake a comprehensive redesign of the frontend architecture, navigation and interaction system. Adding animation to the existing pages will not resolve fragmented navigation, duplicated code, inconsistent state handling or integration gaps. Build a reusable application shell and domain modules, then migrate working business flows in controlled releases.

Before removing any function, page or API, record its users, permissions, business rules, dependencies and replacement. Preserve existing behavior until the replacement passes functional, authorization and reconciliation tests. The first milestone should deliver login, a permission-aware launcher, responsive navigation, notifications and profile controls, a shared API client, reusable forms and tables, and one Inventory workflow connected to Laravel.

The intended business scope is a textile or linen manufacturer operating one factory and five showrooms. Manufacturing, stock, showroom sales and finance must remain connected through authoritative backend transactions. Supporting applications include Purchasing, Costing, CRM and Loyalty, HR and Payroll, Delivery, Marketing, Reports and Administration. [S1]

Scope and technical baseline

Retain PHP/Laravel and MySQL as the business backend. Use HTML, Tailwind CSS and JavaScript ES modules for the frontend baseline; a build tool may compile assets without becoming a backend runtime. Adopt a framework only through an explicit architecture decision supported by team capacity and maintainability needs.

The repository declares Laravel 11 and PHP 8.2 or later. It currently has a separate frontend directory, a shared browser API object, custom bearer-token middleware and unversioned /api routes. These are static source observations, not evidence that deployed workflows have passed testing. The proposed architecture and endpoint examples in this report require contract review before implementation. [R1-R4]

Reading guide

- Section 1 describes glassmorphism, the launcher, tiles and responsive interaction standards.
- Section 2 defines file boundaries, routing, component reuse, state and testing architecture.
- Section 3 inventories application pages, workflows and access controls.
- Section 4 defines current-to-target API integration, authentication, payloads and failure states.
- Section 5 defines Laravel services, MySQL domains, transaction workflows, costing and security.
- Section 6 sets out phased delivery, acceptance criteria and production release requirements.

The detailed design values, performance budgets and delivery sequence below are proposed engineering targets. Product owners should approve them during discovery; they are not measurements of the existing system.

1 Modern UI and UX Design

1.1 Restrained glassmorphism

Glassmorphism uses translucent surfaces, background blur, fine borders and soft shadows to suggest layered glass. Apply it to the launcher, top bar and lightweight overlays. Keep tables, checkout totals, payroll and financial forms on opaque surfaces so changing backgrounds cannot reduce readability. Blur affects what is behind an element; a partially transparent background is needed to reveal the effect. [S2]

Use a quiet neutral background with a subtle gradient, one primary brand color, and semantic success, warning and error colors. The existing gold-and-white identity can be retained with dark readable text and restrained gold accents; blue is an alternative proposed in the supplied report, not a required rebrand. Approve one palette before implementation.

- Surface tokens: opaque white for data; approximately 88% white for glass; a subtle neutral border; a low-opacity shadow. Validate actual composited contrast.
- Scale: 4 and 8 px spacing rhythm; 8 px input radius, 12 px cards and 18 px launcher tiles; 14-16 px body text with a clear heading hierarchy.
- Motion: 140-220 ms opacity or transform transitions; a small tile lift and press response; no moving background behind working data.
- Resilience: default to an opaque surface, add blur only through feature detection, and provide a solid-surface preference for performance or readability.

```
.glass-panel { background: #fff; border: 1px solid #d5dce5; }
@supports (backdrop-filter: blur(12px)) {
  .glass-panel {
    background: rgb(255 255 255 / 88%);
    backdrop-filter: blur(12px);
  }
}
.app-tile { transition: transform 160ms ease, opacity 160ms ease; }
.app-tile:hover { transform: translateY(-2px); }
@media (prefers-reduced-motion: reduce) {
  .app-tile { transition: none; }
  .app-tile:hover { transform: none; }
}
```

1.2 Accessibility and interaction

Target WCAG 2.2 AA: normal text contrast of at least 4.5:1, large text 3:1, and appropriate non-text contrast for controls and focus indicators. Meet the 24 by 24 CSS px target-size criterion or its permitted exceptions; use approximately 44 px touch controls as a practical project target. Verify focus visibility and keyboard access at 200% zoom and content reflow at narrow widths. [S3]

Honor reduced-motion preferences by removing decorative movement. [S4] Pair status color with text and an icon, label every input, and expose inline errors to assistive technology. Dialogs must have an accessible name, move focus inside, contain focus while open, support appropriate dismissal and return focus to the initiating control. Financial confirmation dialogs must clearly name the transaction and consequence.

1 Modern UI and UX Design continued

1.3 Modular launcher and application tiles

Use an Odoo-like separation between choosing an application and working inside it. After login, show a home workspace with pinned applications, recent work, pending approvals and role-relevant KPIs. A persistent app-switcher opens the full permitted application grid from every module. Provide a distinct All applications view so operational dashboards do not become crowded icon directories.

Each tile follows the visual consistency of an iPhone app grid: a rounded icon container, recognizable symbol, short name and uniform alignment. Use a web link for navigation, not a clickable anonymous div. Include an optional one-line description and an accessible pending-work badge. Do not copy platform branding or make reordering dependent on drag alone; provide Pin and Move actions accessible by keyboard.

Example tiles are Sales and POS with a shopping-bag symbol, Manufacturing with a factory, Inventory with stacked boxes, CRM with users, HR with an identification card, Finance with a ledger, and Reports with a chart. Use one SVG icon family such as Lucide or Heroicons after license and bundle review. Decorative icons are hidden from screen readers; icon-only controls need accessible names. Cache user preferences, but derive access from the current server identity.

1.4 Shared shell and responsive behavior

- Top bar: current company and branch, app switcher, permission-scoped global search, authorized quick-create actions, notifications, help and profile.
- Sidebar: current module, grouped routes, active-page indication and collapse preference. Use a drawer on small screens; opening and closing it must preserve keyboard focus.
- Content: breadcrumbs, title, description, primary action, filters, records and pagination. Keep primary actions stable between list and detail views.
- Desktop: expanded sidebar and multi-column dashboard. Tablet: compact navigation and two-column cards. Mobile: drawer, single-column tasks and prioritized columns or record cards.
- POS: product search and cart side by side where space permits; a clear cart/checkout step on phones. Preserve barcode-scanner focus and avoid motion during scanning.

Test representative widths of 360, 768, 1024 and 1440 px. These are test points rather than universal device definitions. Let content determine breakpoints. Permit contained horizontal scrolling only where a comparison table genuinely needs two-dimensional layout.

1.5 Notifications and profile management

Notifications distinguish actionable approvals, operational warnings and informational updates. Provide unread counts, mark-as-read, timestamps, preferences and links to authorized records. Deduplicate repeated events; recheck permission when opening a destination. Use a polite live region for routine updates, avoiding a stream of screen-reader interruptions.

Profile pages cover name, avatar, language, timezone, display preferences, password change and session controls. Email or password changes require appropriate reauthentication. Role and branch changes belong in Administration. Start with polling for notification updates; introduce private realtime channels only if latency requirements justify the extra infrastructure.

2 Complete Frontend File Architecture

2.1 Domain ownership and repository layout

Keep the existing frontend/backend boundary during migration. The source PDF proposes Laravel resources/js as an alternative; either layout works, but maintaining both as competing source trees creates drift. The following target adapts domain separation to the actual repository. Asset builds should emit deployable files to dist, with the web server forwarding API traffic to Laravel.

```
frontend/
  index.html
  public/                    static assets and icons
  src/
    main.js                  application bootstrap
    app/
      router.js              route matching and lazy loading
      routes.js              path and permission metadata
      app-registry.js        launcher and module navigation
      guards.js              session and route UX checks
      providers.js           shared services and state wiring
    layouts/                 AuthLayout, AppShell, HomeLayout
    styles/                  tokens.css, app.css, components.css
    components/
      ui/                    Button, Modal, Drawer, GlassPanel
      data/                  DataTable, FilterBar, Pagination
      forms/                 FormField, ErrorSummary, FormActions
      navigation/            AppTile, Sidebar, TopBar, Breadcrumbs
      feedback/              Toast, Skeleton, EmptyState, ErrorState
      charts/                KpiCard, ChartCard, accessible summaries
    modules/
      home/                  pages, components, services, tests
      sales/                 pages, components, services, schemas, tests
      manufacturing/         same module structure
      inventory/             same module structure
      costing/ purchasing/ crm/ hr/ finance/
      delivery/ marketing/ reports/ admin/
    services/                http, auth, uploads, exports, notifications
    stores/                  session, preferences, notification state
    utils/                   money, dates, quantities, errors
    locales/                 language dictionaries
  tests/                     integration, e2e, fixtures, accessibility
  dist/                      generated production assets
  package.json               optional build and test tools
backend/                     Laravel source and domain rules
docs/                        API contracts and architecture decisions
```

Each module exports its routes and deliberate public interfaces. Pages compose components and coordinate requests; services translate contracts; schemas describe client input checks; backend rules remain authoritative. Shared components must not import module pages. A Sales page can link to a customer route through a shared route helper without importing CRM internals.

Keep loyalty under CRM and payroll under HR initially, with subfolders and explicit boundaries. Split them into separate modules when ownership or complexity warrants it. Avoid a generic utility directory becoming a home for business logic. Co-locate module tests and name files by purpose, such as ProductionOrderDetail.js and production-service.js.

2 Architecture continued

2.2 Home structure and routing

HomeDashboard composes WorkQueue, KpiSummary, RecentRecords and PinnedApps. AppLauncher reads an application registry containing id, title, icon, route, required capability and navigation groups. Intersect the registry with server permissions and enabled modules. A central registry keeps the launcher, switcher and sidebar consistent.

Navigation example: login -> home -> Inventory tile -> stock list -> item detail -> transfer form -> submitted transfer detail. Breadcrumbs retain context, and Back returns to the original filtered list. Store filters, sorting and page number in URL query parameters; preserve only non-sensitive preferences locally. A hard refresh of /apps/inventory/items/123 must resolve through the server's SPA fallback, while /api paths must never return index.html.

Use nested paths such as /apps/sales/invoices/:id and lazy-load modules. Include explicit not-found, forbidden and session-expired views. Validate return URLs against internal routes. Warn about unsaved changes before app or branch switching. On branch change, cancel pending requests, clear branch-dependent caches and refetch identity-scoped data.

2.3 Component and state contracts

DataTable accepts columns, records, selection, sort, pagination and explicit loading/error states. It emits user intent rather than making its own domain requests. FormField owns label, help text and error associations. A shared modal supplies focus behavior; a module-specific ReturnDialog supplies reason codes and business text. Toast messages report completed work but never replace persistent form errors or a saved transaction status.

Use three state scopes: server state for records and balances, session state for identity and active scope, and local state for drafts and open panels. Cache server data by user, company, branch, module and normalized filters. Invalidate affected records after committed mutations. Do not optimistically finalize stock, payroll or payments. Clear protected data on logout and permission changes. Use AbortController or request sequence checks to prevent slow search results overwriting newer queries.

2.4 Services and frontend tests

One HTTP client handles base URL, credentials, timeouts, response parsing and normalized errors. Module services own endpoint paths and map legacy responses into stable view models. Components do not duplicate fetch calls. Define contracts in OpenAPI, review schema diffs and use contract fixtures to detect accidental changes.

Unit tests cover money display, error normalization and permission predicates. Component tests verify keyboard behavior, form errors, dialogs and table states. Page integration tests exercise filters, pagination and service failures with controlled responses. Browser tests verify real login, branch switching, checkout and expiry against a test backend. Choose one unit/component runner and one browser runner, such as Vitest and Playwright, after confirming the build environment; test tools do not dictate the production framework.

3 Frontend Pages and Functions

3.1 Sales and POS

Provide a sales overview, terminal and cash-session opening, product/barcode search, cart, customer selector, discount approval, checkout, invoice list and detail, receipt print, payment history, returns/exchanges and daily closing. Checkout supports approved cash/card/credit splits and shows server-calculated totals before finalization. Display processing, confirmed and status-unknown outcomes distinctly. Returns reference original invoice lines and validate remaining returnable quantities; do not edit posted invoices silently.

3.2 Manufacturing and costing

Provide a production dashboard, BOM list and versioned editor, material requirement preview, production order list/detail, scheduling, release/start actions, material issue, actual consumption, wastage, output/batch entry and completion. Show planned versus actual quantities and costs. Required inputs include units, location, BOM version and effective date. Block invalid transitions with an explanation and preserve the user's entries when validation fails.

Costing pages include cost rules, BOM estimates, completed batch cost sheets, overhead allocation, historical cost snapshots and product profitability. Display calculation basis, effective rule and whether figures are estimates, standard costs or actuals. Restrict cost visibility independently from product visibility.

3.3 Inventory and showroom operations

Provide product and variant masters, categories, units and conversions, warehouses and showroom locations, balances, stock movement ledger, transfers, counts, adjustments and approvals. Show on-hand, reserved and available quantities separately. A transfer page records source, destination, dispatch and receipt where physical transit exists. Count approval should explain the variance and create an auditable adjustment rather than directly changing a balance.

Showroom users receive a branch dashboard with local stock, incoming transfers, current POS sessions and authorized targets. A branch selector must never imply access to another location merely because its identifier appears in a URL.

3.4 CRM and loyalty

Provide customer search, profile and addresses, contact notes, purchase history, credit account, outstanding invoices, collections, loyalty ledger, tiers and rewards. Record consent and restrict sensitive contact exports. Credit-limit changes and manual loyalty adjustments require permission and a reason. Product availability and customer credit shown in a form are advisory until the backend validates the final transaction.

3.5 HR and payroll

Provide employee profiles, departments, attendance, leave, overtime, salary structures, payroll periods, draft calculation, review, approval, payslips and commission review. Separate employee self-service from HR administration. Explain calculation components and approval status without exposing another employee's salary. Submitted and finalized payroll require controlled transitions and adjustment records. Jurisdiction-specific payroll and statutory rules need business and specialist approval before implementation.

3 Pages and Functions continued

3.6 Finance and reports

Finance pages include expenses and approvals, cash sessions, receivables and aging, payables, customer/supplier allocations, account balances, journals and period close where a full ledger is in scope. If the existing system supplies only operational summaries, label them accordingly; a complete general ledger is a separate backend deliverable. Provide attachments, references, reasons for reversals and drill-down to source transactions.

Reports include sales by branch/product/cashier, stock balance and movement, production yield and wastage, costing and profitability, debt aging, purchasing, payroll and commissions, delivery/fuel, campaign spending and financial summaries. Each report shows date range, timezone, branch scope, included statuses, calculation basis and data freshness. Export jobs reproduce the same scope and filters and provide a completion notification and expiring download.

3.7 Supporting applications

- Purchasing: supplier master, purchase orders and approvals, receipts, supplier invoices, payment allocations and outstanding balances. Support partial receipts and remaining quantities.
- Delivery and Fleet: delivery orders, dispatch and completion, drivers, vehicles, proof of delivery, fuel, maintenance and transport cost. Limit personal data visible to drivers.
- Marketing: campaigns, budgets, expenses and measured outcomes. Distinguish tracked attribution from estimates and avoid claiming causal revenue from incomplete data.
- Administration: users, roles, capabilities, branch assignments, numbering rules, UI settings, approval limits and audit search. Preview access changes before saving and audit privileged actions.

3.8 Common list and form behavior

Use server-side search, filter, sort and pagination for large lists. Debounce text input around 250-350 ms, cancel superseded requests and reset to page one when filters change. Whitelist sortable fields on the backend, use a stable tie-breaker such as id, cap page size and disclose whether bulk selection means this page or every filtered result. Prefer cursor pagination for rapidly changing large ledgers; use numbered pages where totals and random access are useful.

Forms need visible labels, required indicators, defaults, help text, field errors and an error summary linked to fields. Preserve draft inputs on correctable errors. Use explicit Save draft, Submit, Approve, Post and Reverse actions. Complex multi-line documents use full pages; short quick edits may use drawers. Report generation runs asynchronously when it cannot complete within the normal request budget.

3.9 Role-aware navigation

Illustrative capabilities: cashier can create sales within an assigned showroom; storekeeper can transfer stock; factory supervisor can complete production; HR officer can manage employees; finance approver can approve expenses. Export, reversal, discount override and role administration are separate grants. Hiding controls improves usability, while backend policies must authorize every record, action and scope. A 403 page offers a safe return path without leaking inaccessible record details. [S6]

4 Backend API Integration

4.1 Existing interfaces and proposed contracts

The following existing paths are declared under /api in the inspected source. Route presence does not establish correctness, coverage or live availability. Current generic master-data routes also require review of their allowlists and permission checks. [R2]

- Identity: POST /api/auth/login, GET /api/auth/me, POST /api/auth/logout.
- Home: GET /api/dashboard, GET /api/settings, POST /api/alerts/read.
- Inventory: POST /api/inventory/transfers; POST /api/inventory/adjustments; POST /api/inventory/adjustments/{id}/approve or /reject.
- Manufacturing: POST /api/boms, PUT /api/boms/{id}, GET /api/boms/{id}/cost; POST /api/production-orders; POST /api/production-orders/{id}/release, /start, /complete or /cancel.
- Sales: GET /api/pos/catalog, POST /api/pos/checkout, POST /api/invoices/{id}/return; POST /api/cashier-sessions/open and /api/cashier-sessions/{id}/close.
- Purchasing: POST /api/purchase-orders, /api/purchase-orders/{id}/approve, /api/purchase-orders/{id}/receive and /api/supplier-invoices/{id}/pay.
- HR and Finance: POST /api/payroll-periods; GET /api/payroll-periods/{id}; POST generate/advance actions under that period; POST /api/expenses and approval/rejection actions; GET /api/reports/{name}.

Adopt /api/v1 as a proposed versioned contract, with an adapter or explicit new routes. Merely changing the frontend prefix will break the current API. Preserve old consumers until a documented deprecation gate is passed.

4.2 Target endpoint conventions

Use GET for list/detail, POST for creation or workflow commands, PATCH for partial draft updates and DELETE only for eligible unposted resources. Successful creation returns 201; reads/updates typically return 200; deletion can return 204 with no JSON body. Existing PUT clients must remain supported until migrated.

- Platform: GET /api/v1/me, /apps, /dashboard/summary and /notifications; PATCH /api/v1/me/preferences and /notifications/{id}/read.
- Inventory: GET /api/v1/products, /inventory/balances and /inventory/movements; POST /api/v1/stock-transfers; POST /stock-transfers/{id}/dispatch or /receive under the same prefix.
- Production: GET/POST /api/v1/production-orders; POST /api/v1/production-orders/{id}/issue-materials or /complete; GET /api/v1/products/{id}/cost-sheets.
- Sales and CRM: POST /api/v1/sales/checkouts; GET /api/v1/sales/invoices/{id}; POST /api/v1/sales/returns; GET/POST /api/v1/customers.
- HR, Finance and Reports: POST /api/v1/payroll-runs/{id}/approve; GET /api/v1/receivables and /payables; POST /api/v1/exports; GET /api/v1/exports/{id}.

For each endpoint document required permission, branch scope, schema, status codes, pagination, concurrency behavior and examples. Add missing detail or action endpoints only after confirming a real workflow need.

4 Authentication and Data Contracts

4.3 Authentication migration

The existing login returns a bearer token and user payload; the browser stores them in localStorage. The backend stores a SHA-256 token hash with a 12-hour expiry and one token field per user. This is custom authentication, not an installed Sanctum SPA flow. The current /auth/me response uses a user property, not the proposed data envelope. Preserve these differences in the transition adapter. [R3-R4]

For the first-party browser app, prefer a reviewed migration to Laravel Sanctum session-cookie authentication on the same site. Sanctum's SPA mode uses sessions and CSRF protection rather than issuing the SPA a personal access token. Its documented SPA deployment requires the frontend and API to share a top-level domain. Configure stateful origins, credentials and cookie attributes explicitly. [S5]

The target sequence is GET /sanctum/csrf-cookie, POST the configured login route with credentials and CSRF header, then GET the identity endpoint. This sequence must not be pointed at the current token-only controller without backend changes. Prefer a same-origin reverse proxy. For genuinely separate-site clients, choose a supported token or OAuth architecture deliberately. Never expose server secrets in the asset bundle.

4.4 Request and response example

Illustrative target request: POST /api/v1/sales/checkouts with Accept: application/json, Content-Type: application/json and an Idempotency-Key generated once for that checkout attempt. Cookie-authenticated mutations also send the configured CSRF token. Numbers representing money or fractional quantities are decimal strings in the contract.

```
{
  "branch_id": 2,
  "customer_id": 45,
  "lines": [{"product_id": 101, "quantity": "2.000"}],
  "payments": [{"method": "cash", "amount": "6000.00"}],
  "currency": "LKR"
}

{
  "data": {"id": 784, "status": "posted", "total": "6000.00"},
  "meta": {"request_id": "req-example"}
}
```

The server resolves allowed prices, discounts, tax, credit limits and stock; it must not trust a client total. The values above are synthetic contract examples. Currency and tax configuration must match approved company settings.

Paginated responses use data plus meta.current_page, per_page, total and last_page, and links.next/prev. Laravel API Resources can provide a stable serialization boundary and pagination metadata; adapters must reconcile legacy shapes. [S7] Use ISO 8601 timestamps with offsets, store instants consistently, and distinguish local business dates from timestamps. Include version identifiers for concurrently editable drafts.

4 Errors States and Data Flow

4.5 Validation and error handling

Client validation gives immediate feedback; Laravel Form Requests or equivalent server validation enforce the contract. Laravel returns 422 for JSON validation failures with field-indexed errors. [S8] Preserve paths such as lines.0.quantity so the page can focus the exact invalid field. Current business-rule exceptions also return 422 with a message only; the adapter must support both forms. [R2]

```
{
  "message": "The submitted data is invalid.",
  "errors": {"lines.0.quantity": ["Quantity must be positive."]},
  "code": "VALIDATION_FAILED",
  "request_id": "req-example"
}
```

- Loading: show a skeleton for first load; retain existing rows with a refresh indicator for subsequent fetches; prevent duplicate submission while a command is pending.
- Empty: distinguish no records yet from no filter matches. Offer an authorized Create action or Clear filters, respectively.
- 401: clear protected caches, request login and retain a safe internal return route. Never automatically replay a payment after reauthentication.
- 403: explain insufficient access; do not loop through login. 404: show a neutral missing-or-unavailable record state.
- 409: proposed conflict response for stale versions or competing stock changes; refresh and let the user review. Existing 422 business conflicts require mapping until migrated.
- 419 in a cookie deployment: recover CSRF/session context with a bounded attempt; replay sensitive mutations only if idempotency makes it safe.
- 429: honor Retry-After and explain delay. Network/5xx: show retry for reads; for writes, first determine whether the server committed the operation.

Normalize errors as status, code, message, fields, requestId and retryAfter. Handle 204 without JSON parsing; distinguish cancellation from failure. Never surface SQL, stack traces, tokens or internal filesystem paths.

4.6 End-to-end data flow

```
User intent -> Page form -> Module service -> Shared HTTP client
  -> Laravel route and authentication -> Record/scope authorization
  -> Validation -> Domain action -> MySQL transaction
  -> Committed resource response -> Cache invalidation -> Updated UI
                          |
                          +-> Outbox/job -> notification or export
```

Use a transactionally stored idempotency key scoped to actor/company and operation, with a payload fingerprint, processing status and durable result reference. The same key and payload return the original result; a changed payload is rejected. Provide an authenticated operation-status lookup for timed-out checkout. GET requests may use bounded backoff; unsafe commands must not be blindly retried. Queue notifications after commit, and use an outbox plus idempotent consumers when reliable delivery is required.

5 Backend and Database Architecture

5.1 Laravel modular monolith

Retain a modular monolith so closely related stock and financial changes can share a database transaction. Separate HTTP orchestration from domain actions. Controllers authorize and validate, invoke actions and return resources. Services own business rules, state transitions and coordinated persistence. Models expose relations and persistence behavior without becoming an unstructured collection of cross-domain workflows.

```
backend/app/
  Http/Controllers/Api/V1/    request orchestration
  Http/Requests/              input validation
  Http/Resources/             public response mapping
  Policies/                  record and action authorization
  Domains/
    Inventory/Actions/       TransferStock, ApproveAdjustment
    Manufacturing/Actions/   IssueMaterials, CompleteProduction
    Sales/Actions/           PostCheckout, ReturnInvoiceItems
    Finance/Actions/         PostJournal, AllocatePayment
    Costing/ HR/ CRM/ Purchasing/ Delivery/ Marketing/
  Models/                    shared relational models
  Jobs/ Events/ Listeners/    background work and integration
  Reports/                   scoped query and export services
  Support/                   decimal arithmetic and identifiers
backend/database/            migrations, factories and seeders
backend/tests/               Unit, Feature and Integration
```

A Domains tree is a target organizational choice, not a requirement to relocate every working class immediately. Existing Services may be retained behind stable interfaces. Use Laravel transactions for multi-record operations; keep external network calls outside long-held database locks. [S9]

5.2 MySQL domain model

- Organization/access: companies, branches, locations, users, role assignments, permission grants and branch memberships.
- Catalog: products, variants, units, conversion rules, categories, price lists and effective prices.
- Inventory: stock movements, balances, reservations, transfers/lines, stock counts and adjustments. Each movement references its originating business transaction.
- Manufacturing: BOM versions/items, production orders, material issues, outputs, wastage and batch cost snapshots.
- Commercial: suppliers, purchase orders/lines, receipts, supplier invoices; customers, sales invoices/lines, payments, allocations, returns and loyalty ledger.
- People: employees, attendance, leave, pay structures, payroll runs/items and commission records.
- Finance: accounts, journal headers/lines if in scope, fiscal periods, expenses, receivables, payables and reconciliation records.
- Supporting: delivery orders, fleet/fuel records, campaigns, export jobs, audit events, idempotency records and an outbox.

Use InnoDB, foreign keys and tenant-aware unique constraints. Index frequent scope/date/status and item/location queries after examining query plans. Separate reporting projections from transactional truth; disclose projection freshness. Avoid premature separate databases for every module.

5 Workflow and Transaction Integrity

5.3 Data integrity controls

Use DECIMAL with approved precision for quantities and money and decimal-safe PHP arithmetic. Define rounding by currency and business operation; formatting in JavaScript is not accounting. Store original unit, conversion and cost basis where historical reconstruction matters. Use unique document numbers within their intended company/period scope and foreign keys to prevent orphan lines.

Maintain an append-only posted movement ledger; balances are a controlled projection updated transactionally and reconciled to movements. Lock affected stock rows in a consistent order during posting and recheck availability inside the transaction. Define whether negative stock is prohibited, reserved or explicitly approved. Add a version check to editable documents to prevent one user's save overwriting another's.

5.4 Manufacturing to showroom to finance

1. Purchasing receives approved raw materials into a factory location with receipt references and valuation inputs. Supplier liability is posted at the approved accounting recognition point.
2. A production order captures the approved BOM version, planned output and planned cost. Release validates readiness; actual material issue reduces raw stock and records work in progress.
3. Production records actual usage, wastage, labor and output. Completion creates finished-goods movements and an immutable cost snapshot. Wastage treatment and abnormal-loss handling follow approved policy.
4. Showroom transfer dispatch reduces factory availability and places stock in transit. Receipt moves it into the showroom; partial receipt and discrepancy flows remain explicit. An immediate internal relocation may post both sides atomically, but a physical journey must not pretend receipt already occurred.
5. POS validates the cashier's session, branch, prices, discounts, stock and payment/credit split. Finalization posts invoice, stock issue, payment or receivable and historical cost references in one coordinated backend operation.
6. Finance reconciles cash/card settlement, customer credit and source transactions. Returns use a linked credit/reversal and a disposition decision for resalable or damaged stock.

Where a full general ledger is implemented, illustrative postings are raw material to work in progress on issue, work in progress to finished goods on completion, cash/receivable to sales and tax liability on sale, and cost of goods sold to finished goods for the stock issue. Final account mappings and tax treatment need accounting approval; these examples are architecture illustrations rather than jurisdiction-specific advice.

5.5 Payments and reliability

An external card provider cannot participate in the local MySQL transaction. Model authorization, capture, failed and pending outcomes, verify signed callbacks, and deduplicate provider events. Reconcile unmatched payments and invoices through an operational queue. Do not hold a database transaction open while waiting for a terminal or network call. Offline POS is a separate project requiring reservation, conflict and payment rules; the initial redesign should make connectivity loss explicit and prevent unsupported offline posting.

5 Costing Profitability and Security

5.6 Costing example and analysis

Manufacturing cost comprises direct material, direct labor, production packaging, allocated factory overhead and other approved production costs. Choose and document an inventory valuation method such as FIFO or weighted average, the treatment of standard/actual variances and the costing date. Record cost versions and allocation drivers so later rule changes cannot rewrite posted history.

Illustrative batch: material LKR 120,000 + labor 30,000 + packaging 10,000 + approved overhead 20,000 = LKR 180,000. If 100 good units are produced and the approved policy absorbs normal loss into good output, unit cost is LKR 1,800. Different treatment is required for abnormal waste; do not divide by planned output when actual usable output differs.

Illustrative sale: net revenue excluding collected tax is LKR 3,000 per unit, historical cost is 1,800, selling expense is 150, delivery allocation is 100 and commission is 90. Gross profit is 1,200 and gross margin is 40%. Management contribution after those specified costs is 860, or 28.67% of net revenue. This is not automatically company net profit: unallocated overhead, finance costs and other items may remain. These figures are synthetic examples.

Reports should show estimated versus actual costs, returns/discount adjustments, quantity, cost snapshot, allocation method and reconciliation to invoices or journals. Require finance approval for shared delivery, marketing and overhead allocation rules. Prevent double counting costs already absorbed into manufacturing cost. Display unavailable cost as incomplete data rather than zero cost or inflated profit.

5.7 Security permissions and audit

Use least privilege across view/create/edit/approve/post/reverse/export operations and company, branch and record scope. Enforce segregation of duties where required, for example preventing a user from approving their own adjustment above a threshold. Policies and gates provide Laravel authorization mechanisms, but scope constraints also belong in queries and domain checks. [S6]

Use HTTPS, a restrictive content security policy, safe text rendering, allowlisted production API origins and secure session settings. Avoid accepting arbitrary API hosts in the production login UI. Validate upload type and size, store sensitive documents privately, and authorize every download. Rate-limit authentication and expensive exports. Never log credentials, full payment details or sensitive payroll payloads.

Audit actor, effective scope, time, action, entity, prior/new approved values, reason, correlation id and source transaction. Redact sensitive fields. Limit audit modification to dedicated system processes, control read access, establish retention rules and export tamper-evident copies where required. Audit role changes, approvals, posting, reversals, cost-rule edits and sensitive exports; an ordinary application error log is not an audit trail.

The repository targets Laravel 11. Its official security-fix window ended on 12 March 2026. Plan and validate an upgrade to a supported release as a release dependency; do not assume the UI redesign fixes backend maintenance exposure. [S10]

6 Implementation Plan for the Development Team

6.1 First steps and audit gate

Start with a joint product/backend/frontend/QA discovery session. Inventory every screen, route, API call, scheduled job, export, permission and external integration. Capture real user journeys with a cashier, storekeeper, production supervisor, HR user and finance reviewer. Record current failures separately from intended behavior so defects are not accidentally treated as contractual requirements.

Create a migration register with old screen/function, consumer, endpoint/method, payload, role/branch scope, side effects, owner, usage evidence, replacement, tests and retirement decision. Example: current checkout uses POST /api/pos/checkout; map it to the proposed sales service, record pricing/payment rules, verify duplicate-request handling and reconcile resulting stock and receivables before switching traffic.

Run route:list in a configured environment, inspect generic resource allowlists, capture sanitized contract fixtures, and map tables to business events. Do not remove an apparently unused API without checking scheduled jobs, reports, integrations and older clients. Complete a backup and restore rehearsal before schema changes.

6.2 Phased delivery and exit gates

- Phase 1, discovery and contracts: product lead and backend lead approve the function map, role matrix, valuation decisions, API gaps and migration register. Estimate delivery only after this inventory.
- Phase 2, UX and foundation: design/frontend leads deliver tokens, launcher, shell, profile, notifications, routing, shared client and components. Exit when keyboard/responsive tests and authentication states pass.
- Phase 3, one vertical slice: Inventory products, scoped balances, movement history and a controlled transfer connect end to end. Exit when UI, API, permissions and transaction effects reconcile with test fixtures.
- Phase 4, supply and production: Purchasing, Manufacturing and Costing add receipts, BOMs, consumption, output and historical costs. Exit after partial receipts, wastage and completion scenarios pass.
- Phase 5, retail: Sales/POS and CRM/Loyalty add sessions, checkout, returns, credit and customer history. Exit after concurrency, payment ambiguity and daily close tests pass.
- Phase 6, remaining domains: HR/Payroll, Finance, Delivery, Marketing, Reports and Administration migrate against approved rules. Exit with source-to-report reconciliation and data-access sign-off.
- Phase 7, pilot and release: a controlled showroom/user cohort validates complete operations. Expand only after agreed support, error and reconciliation thresholds are met.

Use feature flags or route-level switching for coexistence. Avoid two independent writers implementing different business rules; both old and new screens should call compatible backend services during transition. Keep the database compatible with the previous release using expand-and-contract migrations. Retire old functions only after replacement acceptance, consumer checks and the rollback window have passed.

6 Testing Acceptance and Production Release

6.3 Testing and measurable acceptance

Build a traceable test matrix linking each audited function to its API contract, role, branch scope and acceptance evidence. Use synthetic fixtures in an isolated environment. A passing visual review is necessary but does not prove financial or stock correctness.

- UI acceptance: all permitted apps are reachable from the launcher and direct URLs; unauthorized apps are excluded; refresh/back navigation and branch switching work. Shared forms/tables render loading, empty, filtered-empty, validation and error states consistently.
- Accessibility: complete critical workflows by keyboard; verify dialog focus and screen-reader labels; validate contrast, zoom/reflow and reduced motion. Use automated checks plus manual task testing.
- API acceptance: every migrated page uses a documented contract; malformed payloads and unauthorized object identifiers fail safely; 401/403/409/422/429 and network scenarios have tested behavior.
- Integrity: two simultaneous sales of the last unit cannot both succeed under a no-negative-stock policy. Repeating the same checkout key produces one result. Failed posting rolls back all local changes. Returns cannot exceed original remaining quantities.
- Reconciliation: stock balances equal scoped movement totals; payment allocations reconcile to outstanding balances; a full-ledger implementation balances debits and credits. Test rounding, partial fulfillment, reversals and cost snapshots.
- Security: direct API calls cannot cross branch/company boundaries, elevate privileges, read payroll without rights or export unauthorized data. Test logout, expiry and privilege revocation against cached screens.

Proposed performance budgets on an agreed branch network and representative data volume: initial usable launcher within 2.5 seconds, common list API p95 within 500 ms, and local barcode/cart response within 100 ms. Measure browser and API latency separately; external card authorization has its own budget. Revise targets using the audit baseline and record dataset size, device, concurrency and network conditions.

6.4 Production release checklist

Require product-owner UAT approval and no unresolved critical stock, payment, authorization or data-loss defects. CI must pass relevant frontend, backend, contract and end-to-end checks. Validate supported runtime versions, production builds, environment configuration, migrations, queues, scheduled jobs, private storage and monitoring. Load-test realistic concurrent branch activity and exercise queue/provider failures.

Rehearse backup restoration, schema compatibility and application rollback in staging. Define rollout owner, support owner, rollback trigger and communication plan. Roll back the application with compatible schemas where possible; do not restore an old database over valid new sales as a routine rollback. Reconcile transactions created during any incident.

Monitor API errors/latency, failed jobs, authentication failures, checkout ambiguity, payment reconciliation and stock variances. Pilot with a limited cohort, compare outcomes with the approved baseline and expand gradually. The completion gate is a usable, integrated and reconciled application suite with signed-off replacements for audited functions, not simply a visually refreshed dashboard.

References and Decision Register

Source and repository evidence

[S1] Integrated ERP Frontend Backend UI Architecture Report, supplied PDF, version 1.0, 1 November 2026, 11 pages. Used for business scope and design objectives; its recommendations are not evidence of implemented behavior.

[R1] Repository README.md and backend/composer.json: separate frontend/backend structure; Laravel ^11.0 and PHP ^8.2 declarations.

[R2] backend/routes/api.php and backend/bootstrap/app.php: /api prefix, route declarations, permission middleware and current 422 business-error mapping.

[R3] frontend/assets/js/api.js: current localStorage token/user storage, shared API object and existing error behavior.

[R4] backend/app/Http/Controllers/AuthController.php and backend/app/Http/Middleware/ApiTokenAuth.php: custom token issuance, hashing, expiry and identity response. Source inspection is limited to the report's integration baseline; deployed behavior still needs audit and testing.

Primary technical references

References checked 1 November 2026. Laravel 12 documentation below supports proposed patterns; use the documentation matching the release selected for implementation.

[S2] MDN backdrop-filter reference: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/backdrop-filter

[S3] W3C Web Content Accessibility Guidelines 2.2: https://www.w3.org/TR/WCAG22/

[S4] MDN prefers-reduced-motion reference: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-reduced-motion

[S5] Laravel Sanctum SPA authentication: https://laravel.com/docs/11.x/sanctum#spa-authentication

[S6] Laravel authorization: https://laravel.com/docs/12.x/authorization

[S7] Laravel API Resources: https://laravel.com/docs/12.x/eloquent-resources

[S8] Laravel validation: https://laravel.com/docs/12.x/validation

[S9] Laravel database transactions: https://laravel.com/docs/12.x/database#database-transactions

[S10] Laravel 11 support policy: https://laravel.com/docs/11.x/releases#support-policy

Decisions required before implementation

Record an owner, due date, decision and rationale for: brand palette; build strategy; supported Laravel upgrade; authentication and hosting topology; company/branch scope; approval limits; unit conversions; valuation and wastage rules; taxes and rounding; payment and credit policy; payroll rules; full general-ledger scope; notification latency; report definitions; data retention; offline POS scope; performance budgets; and pilot/rollback thresholds.

The development team's first deliverable should be the approved audit and migration register. Its next should be the shared foundation and one verified module. This sequence makes the complete frontend redesign reviewable while protecting operational continuity.